

BXP Development Journey

How the Breathe Exposure Protocol Was Created

Author: Elvarin

Project span: February 2026 – present

Introduction

This document traces the actual development history of BXP — how the idea began, how it evolved, what problems were encountered, and how the project reached its current state. The goal is to show intellectual development honestly: the idea did not arrive fully formed. It evolved through observation, investigation, realisation, and iteration.

Dates are drawn from the project changelog, file timestamps, and documented development notes. Where exact dates are not recorded, this is stated clearly.

Stage 1 — The Original Question

Date: approximately early February 2026

The project began with a simple personal observation about air quality data. The question was roughly: why is it so hard to get consistent air quality information across different cities and different apps?

This led to investigation of how air quality data is currently collected, stored, and distributed. The initial expectation was that some standard already existed — a universal format that any sensor or application could use.

That expectation turned out to be wrong.

Stage 2 — Discovering the Fragmentation Problem

Date: February 2026, prior to v1.0

Investigation revealed that air quality data exists in a state of significant fragmentation:

- * The US EPA's Air Quality Index (AQI) is a national standard, not an international one
- * European monitoring networks use different indices (CAQI, EAQI) that are not directly comparable with AQI
- * Consumer sensor manufacturers each use proprietary APIs and data formats
- * Research databases are not structured to be interoperable with real-time monitoring systems
- * Open data platforms like OpenAQ aggregate data but do not define a standard format at source

The key realisation was: this is not primarily a measurement problem. It is a data representation problem. The sensors exist. The data exists. But there is no common language for the data to be written in.

This is the interoperability problem that BXP was designed to solve.

Stage 3 — The First BXP Concept (v1.0)

Date: February 15, 2026

The first version of BXP was conceived and initially implemented on February 15, 2026. This date is recorded in the official specification (SPEC.md, Origin & Authorship section).

BXP v1.0 established the foundational concepts:

- * User-owned data — exposure records belong to the person or device that generated them
- * Offline-first architecture — readings can be created without a network connection and submitted later
- * Portable containers — a .bxp file is a self-contained, portable record
- * SHA-256 verification — every record includes a cryptographic hash for integrity checking

The v1.0 implementation was a working Python prototype demonstrating the container concept — a JSON structure representing one or more readings with a payload hash for verification.

Stage 4 — Identifying the Gaps (Pre-v2.0 Development Notes)

Date: February–March 2026

During development, a systematic audit of the v1.0 approach identified 50 specific gaps across six categories. These are documented in the project's development notes and represent the intellectual work of understanding what a complete protocol would need to include.

The major categories identified were:

Protocol completeness gaps:

- * No persistent database — server restarts wiped all submitted data
- * No device token authentication — anyone could write to the node
- * HTTP status codes incorrect (returning 200 where spec required 201)
- * Duration and population factors missing from HRI calculation
- * No geohash precision validation

Unimplemented specification sections:

- * Privacy framework (SHA-256 hashed person IDs, geohash floors, k-anonymity, cryptographic deletion)
- * DELETE /readings/{id} with cryptographic proof
- * GET /readings/{id}/verify — integrity check
- * Community reports
- * Device registration
- * Search endpoint

SDK gaps:

- * No async HTTP client
- * No offline queue for IoT sensors

- * No TypeScript/JavaScript SDK
- * pip install not yet working

Server quality gaps:

- * No rate limiting
- * No structured logging
- * No input validation

Dashboard gaps:

- * No map view
- * No time-series charts
- * No multi-city comparison

Ecosystem gaps:

- * No MQTT bridge (most air sensors use MQTT)
- * No Docker deployment
- * No Postman collection

This audit process — systematically enumerating what a production-quality protocol implementation requires — was itself a significant intellectual exercise. It transformed the question from "is this idea valid?" to "what does a complete implementation of this idea look like?"

Stage 5 — Protocol Expansion and v2.0 Specification

Date: March 2026

BXP v2.0 was the result of the gap analysis in Stage 4. It represented a substantial architectural expansion of the original v1 concept.

Key additions in the v2.0 specification:

- * Complete volume file system architecture (9 top-level directories, logical structure for any storage substrate)
- * Binary .bpx format with 32-byte fixed header
- * Agent schema: 31 atmospheric agents across 7 categories (previously limited set)
- * Five protocol stages (LOCATE → DETECT → INTERPRET → PROTECT → REPORT)
- * Complete REST API specification (15 endpoints with full schema)
- * Security and privacy framework (AES-256-GCM, Ed25519, SHA-256 Merkle tree, k-anonymisation)
- * Community reporting layer with 12 standardised event tags
- * BXP_HRI with duration and vulnerability factors
- * Governance and versioning model
- * Regulatory compliance framework (GDPR, CCPA, POPIA, HIPAA)
- * Compatibility matrix with existing standards (WHO AQG, HL7 FHIR, OGC SensorThings, US AQI)

The CHANGELOG documents this as "The Official Standard" — the move from a working prototype to a complete formal specification.

The v2.0 specification was submitted to Zenodo and assigned a permanent DOI (<https://doi.org/10.5281/zenodo.18906812>).

Stage 6 — Reference Server v2.1 Implementation

Date: 2026 (after v2.0 specification)

Following the v2.0 specification, the reference server was rebuilt from scratch as version 2.1. All the critical gaps identified in Stage 4 were addressed:

Gap identified | Resolution in v2.1
No persistent database | SQLite with structured schema
No device token auth | Full Bearer token validation
POST 201 status | Fixed
Duration/population in HRI | Implemented
No geohash precision validation | Enforced (≥ 5)
No privacy framework | geohash floor, $k \geq 5$ aggregates, crypt
No cryptographic deletion | DELETE endpoint with proof
No integrity check | GET /verify endpoint
No community reports | POST/GET /community/reports
No device registration | POST/GET /devices/register
No search endpoint | GET /search
No rate limiting | Sliding window per IP
No structured logging | Python logging throughout
No input validation | Pydantic models + validators
No pagination | cursor/offset pagination
No map view | Leaflet.js interactive map in dashbo
No time-series charts | Chart.js in dashboard
No multi-city comparison | Dashboard comparison panel
No real-time refresh | 60-second auto-refresh in dashboard
No embeddable widget | GET /widget/{city} iframe
No export | Download .bxp.json from dashboard
No Prometheus metrics | GET /metrics

The reference server grew to approximately 1,877 lines of Python code implementing the full BXP v2.1 node.

Stage 7 — SDK and Tooling Development

Date: 2026 (concurrent with reference server v2.1)

In parallel with the reference server, the Python SDK was substantially expanded to address the gaps identified in Stage 4:

- * write_bxp(), read_bxp(), validate_bxp() — complete file I/O with integrity verification
- * calculate_risk() — full BXP_HRI with duration and population modifiers
- * BXPClient — synchronous HTTP client
- * AsyncBXPClient — asynchronous HTTP client (previously missing)
- * OfflineQueue — local queue with disk persistence (previously missing)
- * encode_geohash() — pure Python implementation (no external dependencies)
- * Full pip packaging (pyproject.toml, setup.py)

The TypeScript SDK was built from scratch as a new addition to the ecosystem.

The CLI tool was substantially expanded to support: batch-submit, CSV/GeoJSON export, HTML map generation, config file management, and BXP_SERVER_URL/BXP_DEVICE_TOKEN environment variables.

Stage 8 — Ecosystem and Infrastructure

Date: 2026

Additional ecosystem components were built:

- * MQTT bridge (integrations/mqtt_bridge.py) — enabling IoT sensors that communicate via MQTT to submit readings to a BXP node automatically
 - * Docker + docker-compose — one-command server deployment
 - * Postman collection — importable API test collection
 - * GitHub Actions CI — automated testing on push
 - * CONTRIBUTING.md — guidance for external contributors
-

Current State (July 2026)

BXP is an independent, open-source research and development project consisting of:

- * A 1,300-line formal protocol specification (BXP v2.0) with permanent DOI
- * A 1,877-line reference server implementation (v2.1)
- * A 895-line Python SDK
- * A TypeScript SDK
- * A 740+-line CLI tool
- * An MQTT bridge
- * A sample dataset covering 10 global cities
- * Complete documentation including API reference, developer guide, architecture, and this evidence package

What remains as future work: binary format implementation, federated node synchronisation, Arduino/ESP32 SDKs, epidemiological review of BXP_HRI, and external adoption.

Lessons Learned

1. Interoperability problems are invisible until you look for them. The fragmentation of air quality data is not widely discussed, but it is structurally present throughout the ecosystem.
 2. A protocol is not the same as an implementation. Designing BXP required thinking about what the protocol needs to guarantee, not just what the first implementation needs to do.
 3. Privacy cannot be bolted on. The privacy architecture had to be designed into the protocol from the start — geohash floors, k-anonymity, and cryptographic deletion are easier to specify before the first line of server code is written than to add later.
 4. Systematic gap analysis is productive. The 50-item audit in Stage 4 transformed a working prototype into a complete protocol. Naming what is missing is as important as building what exists.
 5. Documentation is part of the protocol. A specification that exists only in the developer's head is not a protocol. Writing SPEC.md, the API docs, the developer guide, and this evidence package is part of making BXP real.
-

Future Direction

- * Engage with atmospheric scientists or epidemiologists to review the BXP_HRI weighting scheme
 - * Build the binary .byp encoder/decoder
 - * Implement the federated node synchronisation protocol
 - * Seek adoption by a real sensor network or monitoring organisation
 - * Explore the BXP-HEALTH extension for mapping to HL7 FHIR R4
-

Copyright 2026 Elvarin — Apache 2.0. The air is public. The data should be too.